

Jak pracovat s Gitem ve Visual Studio Code

Průvodce krok za krokem pro úplné začátečníky
verzování překladu knihy Jaiva-dharma — práce s větvemi a mergem

obsahuje instalaci pro macOS i Windows

Obsah

1. Co je Git a k čemu ti bude
2. Vytvoření účtu na GitHubu
3. Instalace Gitu na macOS
4. Instalace Gitu na Windows
5. Základní pojmy, které se budou opakovat
6. Propojení projektu s Gitem (větev main)
7. Nahrání projektu na GitHub (Publish)
8. Nastavení repozitáře: povolit jen merge
9. Práce ve vlastní větvi (feature branch)
10. Uložení a odeslání změn ve větvi
11. Sloučení větve zpět do main (Pull Request + Merge)
12. Stažení aktualizovaného main
13. Prohlížení historie a větví
14. Co dělat, když nastane konflikt
15. Doporučený pracovní postup pro nás dva
16. Řešení běžných potíží
17. Přehled tlačítek a zkratk

1. Co je Git a k čemu ti bude

Git je program, který si o tvém projektu pamatuje celou historii změn. Pokaždé, když prohlásíš „tohle je hotová verze“ (tomu se v Gitu říká **commit**), si Git uloží přesnou podobu všech souborů v tu chvíli. Díky tomu se kdykoliv později můžeš podívat, jak text vypadal dřív, zjistit, kdo a kdy co změnil, a v případě potřeby se ke starší verzi i vrátit.

V této příručce navíc používáme mírně pokročilejší, ale velmi rozšířený způsob práce: veškerá práce se dělá ve vlastní „**větví**“ (**branch**), a teprve hotová a zkontrolovaná práce se sloučí zpátky do hlavní větve **main** pomocí **merge** (sloučení), ne **rebase**. Díky tomu zůstane v historii přesně vidět, kdy která větev vznikla a kdy a jak se sloučila zpátky – historie se nijak nepřepisuje ani nezkracuje.

Pro váš projekt to znamená hlavně tohle:

- Nic se nikdy nemůže nenávratně ztratit nebo přepsat – historie zůstává zachovaná napořád, včetně toho, jak se jednotlivé větve sloučily dohromady.
- Oba dva (ty i tvůj kamarád) můžete pracovat na stejném projektu současně, každý ve své vlastní větvi, aniž byste si vzájemně překáželi v rozpracované práci.
- Větev **main** je vždy ta „oficiální, hotová“ verze – rozpracovaná práce v ní nikdy přímo nevzniká.

Git samotný je program, který běží na tvém počítači. **GitHub** je pak webová služba (zdarma), kam si historii projektu navíc zálohujete online a odkud si ji stahujete ten druhý z vás.

Celý postup se dělá pohodlně přímo ve Visual Studio Code přes tlačítka, jen samotné sloučení větve (merge) budeme dělat na webu GitHubu – i tam jde ale jen o pár kliknutí.

2. Vytvoření účtu na GitHubu

Tento krok je stejný pro macOS i Windows a udělá se jen jednou.

- 1 Jdi na stránku <https://github.com>.
- 2 Klikni na **Sign up**.
- 3 Zadej e-mail, heslo a zvol si uživatelské jméno. GitHub tě provede krátkým ověřením (např. vyřešením jednoduché vizuální hádanky).
- 4 Potvrď svůj e-mail podle instrukcí, které ti GitHub pošle do schránky.

To je vše, účet je zdarma a nemusíš nic dalšího nastavovat. Uživatelské jméno a heslo si zapamatuj – budeš je za chvíli potřebovat ve VS Code.

3. Instalace Gitu na macOS

Pokud jsi už podle předchozí příručky nainstaloval Homebrew (kvůli popplerru), instalace Gitu je stejně jednoduchá – stačí v terminálu spustit:

```
brew install git
```

Pokud Homebrew nemáš, macOS ti Git nabídne nainstalovat automaticky, jakmile ho poprvé zkusíš použít – stačí do terminálu napsat:

```
git --version
```

Pokud se objeví okno s nabídkou nainstalovat „Command Line Developer Tools“, klikni na **Install** a počkej na dokončení. Po instalaci stejný příkaz (**git --version**) ověří, že vše proběhlo v pořádku – měla by se vypsat verze Gitu.

4. Instalace Gitu na Windows

- 1 Jdi na stránku <https://git-scm.com/download/win> – stažení by mělo začít automaticky, případně klikni na odkaz pro 64-bit verzi.
- 2 Spust' stažený instalátor.
- 3 Instalátor má hodně obrazovek s nastavením – ve všech klidně nech přednastavené (výchozí) hodnoty a jen klikej **Next**.
- 4 Na konci klikni **Install** a po dokončení **Finish**.

Aby se Git ve VS Code správně projevil, zavři a znovu otevři VS Code po instalaci.

5. Základní pojmy, které se budou opakovat

Repozitář (repository)	složka projektu, o které si Git pamatuje historii
Commit	„uložený bod“ v historii – snímek podoby všech souborů v daný moment, s krátkým popisem
main	hlavní, „oficiální“ větev projektu – vždy obsahuje hotovou, zkontrolovanou práci
Větev (branch)	samostatná linie vývoje, ve které lze pracovat, aniž by to ovlivnilo main, dokud se sama nesloučí zpět
feature větev	dočasná větev založená z main pro jeden konkrétní úkol (např. jednu kapitolu), po sloučení se dá smazat
Push	odeslání tvých commitů (v aktuální větvi) na GitHub
Pull	stažení commitů, které mezitím udělal ten druhý, k tobě na počítač
Pull Request (PR)	žádost na GitHubu o sloučení jedné větve do druhé, se souhrnem změn ke schválení
Merge	sloučení dvou větví, při kterém se zachová kompletní historie obou (na rozdíl od rebase, který historii přepisuje)
Konflikt	situace, kdy vy dva upravíte stejné místo stejného souboru jinak a Git neví, kterou verzi má vzít

Běžný pracovní cyklus bude ve zkratce vypadat takto: přepnutí na **main** a **Pull** → založení nové **feature větve** → práce na textu → **Commit** a **Push** větve → na GitHubu otevřít **Pull Request** → **Merge** do main → přepnutí zpět na main a **Pull**.

6. Propojení projektu s Gitem (větev main)

Tento krok se dělá jen jednou, na tom počítači, kde projekt jako první vznikl (pravděpodobně u tebe).

- 1 Otevři ve VS Code svou pracovní složku s projektem (**File** → **Open Folder...**), pokud ještě není otevřená.
- 2 Klikni vlevo na ikonu **Source Control** (rozvětřující se čára s kolečky).
- 3 Klikni na modré tlačítko **Initialize Repository**.
- 4 Do pole **Message** napiš popis, například **Počáteční verze projektu**, a klikni na **Commit**.
- 5 Pokud se VS Code zeptá na tvé jméno a e-mail (kvůli podpisu commitů), zadej cokoliv rozumného.

Tím vznikla i hlavní větev, která se automaticky jmenuje **main** – od teď v ní bude vždy jen hotová, zkontrolovaná podoba projektu.

7. Nahrání projektu na GitHub (Publish)

- 1 V panelu **Source Control** klikni na tlačítko **Publish Branch** (případně „Publish to GitHub“).
- 2 VS Code tě požádá o přihlášení ke GitHubu přes prohlížeč – přihlas se a potvrď propojení.
- 3 Na dotaz, zda má být repozitář **veřejný (public)** nebo **soukromý (private)**, zvol vždy **Private**.
- 4 Po chvíli VS Code oznámí, že repozitář **main** byl úspěšně vytvořen a nahrán na GitHub.

Ted' přidej kamaráda jako spolupracovníka:

- 1 Na <https://github.com> otevři svůj repozitář.
- 2 Klikni na **Settings** → **Collaborators**.
- 3 Klikni na **Add people** a zadej jeho GitHub uživatelské jméno nebo e-mail.
- 4 Kamarád dostane pozvánku e-mailem, kterou musí přijmout.

8. Nastavení repozitáře: povolit jen merge

Aby se náhodou nikdy nepoužil **rebase** nebo **squash** (které historii zkracují nebo přepisují), doporučuji si přímo na GitHubu zamknout, že jde slučovat výhradně klasickým mergem. Tento krok se dělá jen jednou.

- 1 Na GitHubu otevři svůj repozitář a klikni na **Settings**.
- 2 V levém menu klikni na **General** a najdi sekci **Pull Requests**.
- 3 Zaškrtnuté nech pouze **Allow merge commits**.
- 4 Odškrtni **Allow squash merging** a **Allow rebase merging**.

Od teď GitHub při slučování žádnou jinou možnost než klasický merge ani nenabídne, takže se nedá nic udělat omylem.

9. Práce ve vlastní větvi (feature branch)

Předtím, než se pusťíš do práce na konkrétní kapitole nebo korektuře, si vždy nejdřív založ novou větev. Doporučené pojmenování je **feature/** a stručný popis, například **feature/kapitola-10** nebo **feature/korektura-3**.

- 1 V levém dolním rohu VS Code klikni na název aktuální větve (hned vedle ikony větvíčky – zpočátku tam bude psáno **main**).
- 2 Nahoře se otevře nabídka – zvol **Create new branch...**
- 3 Napiš název podle vzoru výše, např. **feature/kapitola-10**, a potvrd' **Enter**.
- 4 VS Code tě na tuto novou větev automaticky přepne – v levém dolním rohu teď uvidíš její název místo **main**.

Než větev založíš, ujisti se, že jsi předtím byl na **main** a měl staženou jeho nejaktuálnější verzi (**Pull**, viz kapitola 12) – nová větev totiž vychází z toho, co právě máš na disku.

10. Uložení a odeslání změn ve větví

Zatímco pracuješ ve své feature větví, ukládej postup stejně jako dřív – jen s tím rozdílem, že vše zůstává v této oddělené větví, dokud ji sám nesloučíš zpět.

- 1 Otevři panel **Source Control**.
- 2 Do pole **Message** napiš krátký popis, například **Přeložena kapitola 10**.
- 3 Klikni na **Commit**.
- 4 Klikni na **Publish Branch** (při prvním odeslání této větve), později už jen na **Sync Changes / Push**.

Commitů může být ve větví klidně víc za sebou (např. jeden po každé dokončené části kapitoly) – to nijak nevadí, do main se nakonec sloučí všechny najednou.

11. Sloučení větve zpět do main (Pull Request + Merge)

Až je kapitola nebo korektura hotová a odeslaná (viz předchozí kapitola), zbývá ji sloučit zpátky do **main**. Tohle se dělá na webu GitHubu:

- 1 Jdi na **https://github.com** a otevři svůj repozitář. GitHub obvykle sám nahoře nabídne žluté tlačítko **Compare & pull request** u tvé nedávno odeslané větve – klikni na něj.
- 2 Zkontroluj přehled změn (zeleně přidané, červeně odebrané řádky), napiš krátký popis, co větev obsahuje, a klikni na **Create pull request**.
- 3 Na další stránce se zobrazí tlačítko **Merge pull request** (díky nastavení z kapitoly 8 už nebude možné zvolit nic jiného než klasický merge).
- 4 Klikni na **Merge pull request** a poté na **Confirm merge**.
- 5 Volitelně můžeš kliknout na **Delete branch**, čímž se smaže už nepotřebná feature větev na GitHubu (v historii přesto zůstane úplně vše zachované).

Sloučení (merge) v historii vytvoří zvláštní „slučovací“ commit, který jasně ukazuje, kdy a jaké dvě větve se spojily – to je přesně ten úplný záznam historie, který jsi chtěl zachovat.

12. Stažení aktualizovaného main

Po sloučení přes Pull Request je nový, aktuální main na GitHubu, ale ještě ne na tvém počítači – je potřeba si ho stáhnout:

- 1 V levém dolním rohu VS Code klikni na název aktuální větve a přepni se na **main**.

- 2 Otevři panel **Source Control**, klikni na tři tečky ... a zvol **Pull** (nebo použij tlačítko **Sync Changes**, pokud se zobrazí).
- 3 VS Code stáhne nově sloučené změny a main bude odpovídat tomu, co je teď na GitHubu.

Odsud může klidně hned začít další kolo práce – založením nové feature větve podle kapitoly 9.

13. Prohlížení historie a větví

- 1 Historii jednotlivého souboru zobrazíš kliknutím pravým tlačítkem na něj v levém panelu → **Open Timeline**.
- 2 Pro přehlednější grafické znázornění všech větví a jejich sloučení doporučuji nainstalovat rozšíření **Git Graph** (přes ikonu Extensions, stejně jako v kapitole 9 první příručky) – po instalaci se dole v panelu Source Control objeví tlačítko, které vykreslí celou historii jako graf s barevně odlišenými větvemi.

Pokud bys se opravdu potřeboval vrátit k nějaké starší verzi celého souboru, napiš mi – projdeme si to spolu, ať se nic omylem nepřepíše.

14. Co dělat, když nastane konflikt

Konflikt nastane jen ve chvíli, kdy vy dva upravíte úplně stejné místo stejného souboru jinak. Díky práci v oddělených větvích je to o dost méně časté než při přímé práci na main, ale při slučování (Merge pull request) na GitHubu se přesto objevit může.

Pokud GitHub při pokusu o merge napíše, že „This branch has conflicts that must be resolved“, nic sám neřeš – napiš mi nebo zavolej. Projdeme si konflikt spolu přímo ve VS Code, kde se dá pohodlně vybrat, která verze textu (nebo obě) má zůstat.

15. Doporučený pracovní postup pro nás dva

- 1 Přepni se na **main** a udělej **Pull** (stáhni aktuální stav).
- 2 Založ novou větev podle úkolu, např. **feature/kapitola-11** nebo **feature/korektura-4**.
- 3 Pracuj na textu. Poznámky a nejasnosti při korektuře označuj podle vzoru ****[POZN: ...]**** popsaného v předchozí příručce.
- 4 Průběžně dělej **Commit** s výstižnými popisky (klidně víckrát za práci na jedné kapitole).
- 5 Po dokončení klikni na **Push** (nebo **Publish Branch** při prvním odeslání dané větve).
- 6 Na GitHubu otevři **Pull Request** a slouč ho do main přes **Merge pull request**.
- 7 Přepni se zpátky na **main** a udělej **Pull**, ať máš u sebe nejnovější sloučenou verzi pro další kolo práce.

Domluvte se předem (třeba zprávou), kdo bude ten den pracovat na které kapitole – práce v oddělených větvích sice umožňuje pracovat současně, ale je pořád lepší se předem sladit, aby nevznikaly zbytečné konflikty při slučování.

16. Řešení běžných potíží

Nevidím tlačítko „Create new branch...”

Klikni přesně na název aktuální větve v levém dolním rohu okna VS Code (ne na jinou ikonu vedle) – tím se otevře nabídka se seznamem větví i volbou pro založení nové.

GitHub nenabízí tlačítko „Compare & pull request“

Klikni v repozitáři nahoře na záložku **Pull requests** a ručně na **New pull request**. Jako „base“ zvol **main** a jako „compare“ svou feature větev.

Tlačítko „Merge pull request“ nabízí i jiné možnosti než merge

Znamená to, že nastavení z kapitoly 8 ještě neproběhlo nebo se neuložilo – vrať se k němu a ověř, že jsou zaškrtnuté jen **Allow merge commits**.

Tlačítko Push/Sync hlásí chybu s přihlášením

Pravděpodobně vypršelo přihlášení ke GitHubu ve VS Code. Klikni vlevo dole na ikonu profilu/úctu a přihlas se znovu.

Objeví se barevně zvýrazněný text s „<<<<<<<<“ a „>>>>>>>>“

Toto je konflikt popsáný v kapitole 14. Nic v souboru zatím neupravuj a napiš mi – vyřešíme ho společně.

17. Přehled tlačítek a zkratk

Přepnout / založit větev	název větve vlevo dole → Create new branch...
Uložit rozpracované změny s popiskem	napsat Message + tlačítko Commit
Odeslat větev na GitHub poprvé	tlačítko Publish Branch
Odeslat další změny ve větvi	tlačítko Sync Changes / Push
Otevřít Pull Request	na GitHubu: Compare & pull request → Create pull request
Sloučit větev do main	na GitHubu: Merge pull request → Confirm merge
Stáhnout aktualizovaný main	přepnout na main → Sync Changes / Pull
Zobrazit historii/graf větví	pravé tlačítko na soubor → Open Timeline, nebo rozšíření Git Graph

Na začátku to působí jako hodně nových kroků najednou, ale v praxi se zúží na jeden opakující se cyklus: nová větev → práce a commity → push → Pull Request → merge na GitHubu → pull na main. Tenhle postup zachovává úplnou a přehlednou historii celého projektu.